



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS**: Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS**: Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS**: Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task**: Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates.

What is the fundamental difference between the "State Space" and the "Search Tree"?

The state space is the environment itself, every reachable configuration of the world plus the transitions between them and it exists whether or not any agent ever searches it. In this, that space is the set of legal grid cells: the `grid_size` bounds minus the walls, with `_successors()` defining the edges between the adjacent cells, so a 12×12 grid has at most 144 states, each appearing exactly once. The search tree, by contrast, is the structure the agent builds while searching: its nodes are paths from the start cell, not cells, so the same state can appear in many different nodes cell (2,3) reached via a detour is a different tree node from (2,3) reached directly. That is why the tree can be infinite even though the state space is finite: without a guard, the agent loop right->left->right forever and keep growing new branches over the same set of cells. The reached set in `bfs_search`, `dfs_search` and `ucs_search` is exactly the bridge between the two — it collapses repeated states back down to the state space, converting a potentially infinite tree search into a graph search having 144 cells.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

It solves the redundant paths. Two ways the same encounter can be re-encountered during search. What happens to the agent is, `dfs_search` uses a Last In First Out (LIFO) stack and always expands the deepest node. Strip the reached set out and the failure is immediate and total. It pushes (0,0)'s neighbors, pops one, say (1,0) pushes its neighbors, which include (0,0) again. It pops (0,0), pushes (1,0) again, and oscillates between two cells forever. Because each stack entry carries path + [next], those paths grow by one cell per iteration. The process doesn't just spin it consumes memory linearly and boundlessly until it exhausts RAM. This is the simple DFS failure mode: DFS's one redeeming property is $O(bm)$ space, and cycles destroy exactly that. It would never reach the pellet even when one sits two cells away, because it commits to the first branch and that branch never terminates.

`_formulate_plan()` would simply never re

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

When I ran both on the same map, BFS walked almost straight to the nearest pellet and only went around walls when it had to, while DFS shot off in one direction, hugged the edge of the grid and came back the long way — on my 8x8 test BFS cleared all 5 pellets in 20 steps for a score of 70, but DFS hit the 60-step limit with 3 pellets still left and ended on -35. BFS is optimal here for one simple reason: every move in this grid costs the same, so a node's depth is the same thing as its path cost. Since BFS uses a FIFO queue, it finishes every cell at depth 1 before looking at depth 2, and so on, which means the first time it touches food there cannot be a shorter route to it. DFS throws that away because it uses a LIFO stack: it always expands the cell it pushed most recently, so it commits to one branch and dives as deep as it can, only backtracking at a dead end. It has no idea what a path costs, it only knows what it saw last, and it returns the first route that reaches the goal no matter how long that route is. The winding look also isn't random — my `DELTAS` order means the stack always pops Down first, so DFS keeps driving down, then left, then up, then right. So DFS isn't broken, it just answers a different question: BFS finds the shortest route to food, DFS finds any route.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

A 1000x1000 grid is a million cells with a branching factor of 4, and BFS is $O(b^d)$ in both time and space, so on paper it dies immediately, although because I wrote graph search with a `reached` set, it actually caps out at the size of the state space instead of 4^d , and a Python set holding a million (x, y) tuples is already hundreds of megabytes on its own. The bigger problem in my code is that every frontier entry stores its whole path (`path + [nxt]`), so the BFS frontier is the huge ring of cells at the current depth and each one drags a list behind it; using a parent-pointer dictionary and rebuilding the path at the end would fix that. UCS has the same bottleneck but is heavier still, since it carries a heap, a `best\_cost` dictionary and the `reached` set, and with

equal step costs it just behaves like BFS anyway — which is exactly what I saw, both finishing in 20 steps with a score of 70. DFS is supposed to be the cheap one at $O(bm)$ space instead of exponential, and its bottleneck is depth rather than breadth: it only holds one path plus its siblings rather than a whole level. But that advantage only applies to tree search, and my DFS keeps a `reached` set, so it stores every visited cell too and ends up using just as much memory, you trade the linear space away to get completeness, and dropping the set to win it back just makes DFS loop forever. On top of that, m can be a million on this grid, so even a "linear" DFS path is a million cells long and useless to the agent. In short, BFS runs out of memory because of breadth while at least being correct, and DFS either uses just as much or never terminates, and gives a terrible path even when it works, the real answer at that scale is A^* .

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING